A Minimal Mathematical Definition of a Stateless Event-Condition-Action Rule Engine

A Domain-Independent Reference Semantics for Deterministic Action Selection

Rohin Gosling, 2026

Independent technical note

Status. *This note has not been peer-reviewed. It defines a small, domain-independent semantics for stateless Event–Condition–Action (ECA) rule evaluation. It does not define an execution architecture.*

Abstract

This note defines a stateless ECA rule engine as a pure mathematical action-selection function. An event occurrence consists of an event type and an optional payload. Payloads and condition sets are finite typed partial maps that distinguish a concrete value, an explicit `null`, and an absent key. A condition-side `null` is an explicit wildcard, whereas a payload-side `null` is a present null value. The query retains the matching rules of greatest specificity, resolves any tie with a fixed deterministic selector, and returns one action or the no-action value $\perp$.

Specificity assigns weight two to a concrete condition, one to an explicit-null wildcard, and zero to an omitted key. The score is defined for every condition set and compared only among rules that fully match the current payload. The engine is stateless because, for a fixed rule base and tie selector, its result depends only on the current event occurrence. Event types, parameter domains, conditions, and actions remain otherwise uninterpreted, making the model independent of any application domain or implementation technology.

Keywords: Event–Condition–Action rules; stateless evaluation; predicates; specificity; rule selection; formal semantics

1 Aim and Semantic Boundary

The aim is to formalize the following occurrence-local computation:

> Given one current event occurrence and one fixed finite rule base, return the single action selected by event matching, condition matching, specificity, and deterministic tie-breaking; return $\perp$ if no rule matches.

The model defines **selection**, not action execution. Classical active-database models include transaction coupling and broader execution semantics [1, 4]; event-processing models include temporal and composite-event detection [2, 3]; and reaction-rule frameworks include messaging, knowledge updates, and execution concerns [5]. This note deliberately excludes those features, along with event histories, mutable external state, rule cascades, scheduling, delivery, rollback, and action side effects.

Production-rule semantics provide a closer comparison for the selection stage. Berstel-Da Silva separates rule applicability from eligibility and parameterizes the control strategy within a state-changing rule-program execution semantics [6]. Yen studies specificity as a conflict-resolution relation and distinguishes logical pattern subsumption from syntactic approximations based on condition counts [7]. RIF-PRD standardizes the match-conflict-resolution-act cycle, observes that production systems disagree on the exact specificity ordering, and leaves its final tie-break implementation-specific [8]. The present note adopts a deliberately narrower profile: it fixes equality and explicit-null wildcard matching, makes a three-level weighted score the explicit specificity policy, exposes a deterministic identifier selector as a parameter, and stops before action execution.

The rule base and tie-breaking policy are fixed configuration. An implementation may load or replace that configuration, but configuration management is outside one evaluation.

2 Functions and Predicates

Let

$$\mathbb{B} = \{\mathsf{false}, \mathsf{true}\}, \qquad \mathbb{N}_0 = \{0, 1, 2, \ldots\}. \tag{1}$$

The symbol $\uplus$ denotes disjoint union.

A function

$$f : X \longrightarrow Y \tag{2}$$

assigns exactly one value in Y to every input in X. A predicate on X is a special kind of function whose codomain is $\mathbb{B}$:

$$P : X \longrightarrow \mathbb{B}. \tag{3}$$

By contrast, a partial function $f : X \rightharpoonup Y$ may be undefined for some inputs. This note uses a total option-valued query instead: the distinguished value $\perp$ makes "no action" an explicit result rather than an undefined computation.

Thus every predicate may be viewed as a Boolean-valued function, but a function returning an event, number, rule, or action is not a predicate.

The terminology used in this note is as follows.

ECA concept	**Mathematical object**	**Correct role**
Condition evaluation	Predicate μ	Returns true or false
Specificity	Function σ	Returns a non-negative integer
"More specific than"	Derived predicate	Compares two specificity scores
Tie-breaking	Selector function τ	Returns one rule identifier from a nonempty tied set
Rule	Tuple or record	Associates an event, condition set, and action
Rule query	Option-valued function Q_τ	Returns one action or $\perp$
Query predicate	Predicate Selected_τ	States whether a given action is selected

Accordingly, the phrase "a query predicate returns an action" is not mathematically correct. A query that returns an action is a **function**. A corresponding predicate is valid when it answers a Boolean question such as "does this query select action a?"

Specificity is also a function rather than a predicate. If desired, it induces the predicate

$$\text{MoreSpecific}(c_1, c_2) \quad \Longleftrightarrow \quad \sigma(c_1) > \sigma(c_2). \tag{4}$$

A condition predicate evaluates one condition set against one payload. It does not search for the first condition set and does not choose among rules. Specificity comparison and tie resolution belong to the rule query.

3 Events, Payloads, and Parameters

3.1 Parameter Keys, Values, and Key–Value Pairs

Let K be a set of parameter keys. For each $k \in K$, let V_k be the set of legitimate non-null concrete values for that key. The family $(V_k)_{k \in K}$ permits different keys to have different types. No ordering, arithmetic, or other operation on concrete values is assumed; the core model uses equality only.

Let ν be a distinguished present-null payload marker, with $\nu \notin V_k$ for every k, and define the extended payload domain

$$\overline{V}_k = V_k \uplus \{\nu\}. \tag{5}$$

A parameter binding is a typed key–value pair

$$(k, v), \qquad k \in K, \quad v \in \overline{V}_k. \tag{6}$$

A key may therefore be present with a concrete value, present with ν, or absent. As a practical implementation note, in a JSON payload, the token `null` denotes ν; omission denotes key absence. The marker ν is not a wildcard and does not equal any concrete value.

3.2 Payloads

Let $\mathcal{E}$ be the set of event types. For each event type $e \in \mathcal{E}$, let $K_e \subseteq K$ be the keys permitted by its payload schema.

Definition 3.1 (Payload)**.** *A payload for e is a finite typed partial map p satisfying*

$$\text{dom}(p) \subseteq K_e, \qquad p(k) \in \overline{V}_k \quad \textit{for every } k \in \text{dom}(p). \tag{7}$$

Let $\mathcal{P}_e$ be the set of all such payloads for e, including the empty map, and let

$$\mathcal{P} = \bigcup_{e \in \mathcal{E}} \mathcal{P}_e \tag{8}$$

be the set of payload maps admitted by at least one event schema.

Using a partial map guarantees at most one value for each key. Its graph is the set of parameter key–value pairs carried by the event occurrence.

The empty map $\varnothing$ represents no payload.

3.3 Events and Event Occurrences

An event type e identifies what occurred. An event occurrence includes the current payload, written $x = (e, p)$.

Definition 3.2 (Event occurrence)**.** *The set of event occurrences is*

$$\mathcal{X} = \{(e, p) \mid e \in \mathcal{E} \wedge p \in \mathcal{P}_e\}\,. \tag{9}$$

The occurrence $(e, \varnothing)$ is an event of type e with no payload.

4 Conditions

4.1 Conditions and Condition Sets

Let $*$ be a distinguished explicit wildcard marker, with $* \notin V_k$ for every k, and define the extended condition domain

$$\widehat{V}_k = V_k \uplus \{*\}. \tag{10}$$

An atomic condition is a typed pair

$$(k, q), \qquad k \in K, \quad q \in \widehat{V}_k. \tag{11}$$

Definition 4.1 (Condition set)**.** *A condition set is a finite typed partial map c whose graph is a set of atomic conditions:*

$$c(k) \in \widehat{V}_k \qquad \textit{for every } k \in \mathrm{dom}(c). \tag{12}$$

Let C be the set of all such condition sets. The partial-map representation prevents contradictory duplicate conditions for the same key. A condition set is interpreted conjunctively: all of its present conditions must match.

Condition form for key k	**Mathematical representation**	**Constraint**
Concrete value v	$c(k) = v \in V_k$	Payload must contain the equal concrete value
Explicit `null` wildcard	$c(k) = *$	No value or presence constraint
Omitted	$k \notin \mathrm{dom}(c)$	No value or presence constraint

As a practical implementation note, in a JSON condition set, the token `null` denotes $*$. A display symbol such as – denotes omission in an authoring table, but it is not a condition value. JSON `null` is therefore interpreted by context: it denotes v in a payload and $*$ in a condition set.

4.2 Condition Predicate

Define the condition-matching predicate

$$\mu : C \times \mathcal{P} \longrightarrow \mathbb{B} \tag{13}$$

by

$$\boxed{\mu(c, p) = \mathsf{true} \quad \Longleftrightarrow \quad \forall k \in \mathrm{dom}(c) :\ c(k) = * \ \vee\ (k \in \mathrm{dom}(p) \wedge c(k) = p(k) \in V_k)\,.} \tag{14}$$

This definition has five immediate consequences:

1. A concrete condition matches only an equal concrete payload value.
2. A concrete condition does not match payload `null` or an absent payload key.
3. An explicit-null condition wildcard matches a concrete value, payload `null`, or an absent payload key.
4. An omitted condition key imposes no constraint, and extra payload parameters are ignored.
5. The empty condition set matches every payload, including no payload, and therefore represents an unconditional condition.

The match is all-or-nothing. A condition set with several matching entries and one contradiction does not partially match and cannot compete in rule selection.

4.3 Explicit Null and Key Absence

An explicit-null wildcard and an omitted condition key impose the same matching constraint: none. They remain distinct because the former is a declared member of the condition set and contributes to specificity, whereas the latter is not a member and contributes nothing. The distinction is therefore a selection-policy distinction, not a difference in the set of payloads matched.

Payload-side `null` has a different role. It denotes the present-null marker ν and is not a wildcard. A concrete condition cannot equal ν; an explicit-null wildcard matches it. This minimal language cannot require "the payload key is specifically null." Such a test would need a separate operator rather than overloading the wildcard token.

4.4 Specificity Function

For a condition set c, define its numbers of concrete and explicit-wildcard entries by

$$n_{\text{con}}(c) = |\{k \in \text{dom}(c) \mid c(k) \in V_k\}|, \qquad n_{\text{wild}}(c) = |\{k \in \text{dom}(c) \mid c(k) = *\}|. \tag{15}$$

Definition 4.2 (Specificity)**.** *The specificity of a condition set is*

$$\boxed{\sigma : C \longrightarrow \mathbb{N}_0, \qquad \sigma(c) = 2n_{\text{con}}(c) + n_{\text{wild}}(c).} \tag{16}$$

Thus a concrete condition contributes two, an explicit-null wildcard contributes one, and an omitted key contributes zero. Replacing an omitted key with a wildcard increases specificity by one; replacing that wildcard with a concrete equality increases it by one again.

The score is defined structurally for every condition set, independent of a payload. It is compared only after $\mu(c, p)$ is true, so a nonmatching rule never competes regardless of its score. The weights are an explicit selection policy rather than an inherent logical truth [7]. In particular, because explicit null and omission have the same matching extension, their different scores encode declared authoring priority.

4.5 Null and Absence Scenarios

The following matrix applies the definitions to three keys. In each position, `v` denotes the same legitimate concrete value on both sides, `null` denotes ν in a payload and $*$ in a condition set, and `-` denotes an absent key. The score is shown even when the condition does not match; only scores on rows where $\mu = \mathsf{true}$ are eligible for comparison.

No.	**Payload** (k_0, k_1, k_2)	**Condition** (k_0, k_1, k_2)	μ	σ
1	`(v,v,v)`	`(v,v,v)`	true	6
2	`(null,v,v)`	`(v,v,v)`	false	6
3	`(v,v,v)`	`(null,v,v)`	true	5
4	`(null,v,v)`	`(null,v,v)`	true	5
5	`(v,null,v)`	`(null,v,v)`	false	5
6	`(null,v,v)`	`(v,null,v)`	false	5
7	`(null,null,v)`	`(v,null,v)`	false	5
8	`(null,null,v)`	`(v,v,null)`	false	5
9	`(v,null,v)`	`(null,null,v)`	true	4
10	`(v,v,null)`	`(null,null,v)`	false	4
11	`(null,null,null)`	`(v,v,v)`	false	6
12	`(v,v,v)`	`(null,null,null)`	true	3
13	`(v,v,v)`	`(v,v,v)`	true	6
14	`(-,v,v)`	`(v,v,v)`	false	6
15	`(v,v,v)`	`(-,v,v)`	true	4
16	`(-,v,v)`	`(-,v,v)`	true	4
17	`(v,-,v)`	`(-,v,v)`	false	4
18	`(-,v,v)`	`(v,-,v)`	false	4
19	`(-,-,v)`	`(v,-,v)`	false	4
20	`(-,-,v)`	`(v,v,-)`	false	4
21	`(v,-,v)`	`(-,-,v)`	true	2
22	`(v,v,-)`	`(-,-,v)`	false	2
23	`(-,-,-)`	`(v,v,v)`	false	6
24	`(v,v,v)`	`(-,-,-)`	true	0

5 Actions

Let $\mathcal{A}$ be a set of actions. Actions are abstract values in the mathematical model. Selecting an action does not execute it and does not assign any state-transition or side-effect semantics to it.

Let $\perp \notin \mathcal{A}$ denote that no action was selected, and define

$$\mathcal{A}_\perp = \mathcal{A} \uplus \{\perp\}. \tag{17}$$

The no-action symbol $\perp$ is distinct from every member of $\mathcal{A}$.

6 Rules and the Rule Query

6.1 Rules and Rule Bases

Definition 6.1 (Rule)**.** *Let $\mathcal{I}$ be a set of rule identifiers. A rule is a tuple*

$$r = (i, e, c, a) \in \mathcal{I} \times \mathcal{E} \times \mathcal{C} \times \mathcal{A}, \tag{18}$$

where

- *i is a stable identifier;*
- *e is an event type;*
- *c is a condition set; and*
- *a is an action.*

A rule is well-formed when

$$\{k \in \mathrm{dom}(c) \mid c(k) \in V_k\} \subseteq K_e. \tag{19}$$

Thus every concrete condition key must be permitted by the event schema. An explicit wildcard may name a globally defined key outside that schema because it also matches the key's absence.

Let $\mathcal{R}$ be the set of all well-formed rules.

The identifier has no condition-matching meaning. It distinguishes rules and permits a deterministic tie order.

Let R be a finite set of well-formed rules with unique identifiers. Finiteness guarantees that every nonempty set of matching rules has a greatest specificity score.

Write $\mathrm{id}(r)$, $\mathrm{event}(r)$, $\mathrm{condition}(r)$, and $\mathrm{action}(r)$ for the corresponding rule components.

6.2 Applicable-Rule Predicate

For an occurrence $x = (e, p)$, define

$$\mathrm{Applicable}(r, x) \quad \Longleftrightarrow \quad \mathrm{event}(r) = e \wedge \mu(\mathrm{condition}(r), p) = \mathsf{true}. \tag{20}$$

This is a predicate. The set of applicable rules is

$$L_R(x) = \{r \in R \mid \mathrm{Applicable}(r, x)\}. \tag{21}$$

If $L_R(x) \neq \varnothing$, its maximally specific rules are

$$B_R(x) = \{r \in L_R(x) \mid \forall s \in L_R(x) : \sigma(\mathrm{condition}(r)) \geq \sigma(\mathrm{condition}(s))\}. \tag{22}$$

Thus condition matching is performed first, and specificity is used only to rank the rules that fully match.

6.3 Tie-Breaker Selector

For a set Y, let $\mathcal{F}^+(Y)$ denote its nonempty finite subsets. A tie-breaker selects one stable identifier from the identifiers of the tied rules:

$$\boxed{\tau : \mathcal{F}^+(\mathcal{I}) \longrightarrow \mathcal{I}, \qquad \forall J \in \mathcal{F}^+(\mathcal{I}) : \quad \tau(J) \in J.} \tag{23}$$

For a nonempty rule set S, write

$$\mathrm{ids}(S) = \{\mathrm{id}(r) \mid r \in S\}. \tag{24}$$

The concrete policy may remain implementation-defined, but a fixed deterministic τ is a required parameter of the mathematical engine. Without it, a tied query is set-valued or under-specified and cannot promise one action.

A mathematical set has no first occurrence. A "first rule wins" policy therefore requires an indexed rule sequence or a fixed strict total order $\prec$ on rule identifiers. It may then be defined by

$$\tau(J) = \min_{\prec} J. \tag{25}$$

For $L_R(x) \neq \varnothing$, let $w_{\tau,R}(x)$ be the unique rule in $B_R(x)$ whose identifier is selected:

$$w_{\tau,R}(x) = \text{the unique } r \in B_R(x) \text{ such that } \mathrm{id}(r) = \tau(\mathrm{ids}(B_R(x))) \,. \tag{26}$$

Existence follows from the selector law, and uniqueness follows from unique rule identifiers. The selector is therefore applied only to the maximally specific tied rules. It receives identifiers only and cannot inspect condition contents. If all tied rules name the same action, the returned action is independent of τ; otherwise the selected action depends on the fixed tie policy.

6.4 Query Function

Definition 6.2 (Stateless ECA rule query)**.** *Let $\mathfrak{R}$ be the set of finite, well-formed rule bases with unique identifiers. Define the rule query*

$$Q_\tau : \mathcal{X} \times \mathfrak{R} \longrightarrow \mathcal{A}_\perp \tag{27}$$

by

$$\boxed{Q_\tau(x, R) = \begin{cases} \perp, & L_R(x) = \varnothing, \\ \mathrm{action}\big(w_{\tau,R}(x)\big), & L_R(x) \neq \varnothing. \end{cases}} \tag{28}$$

For a fixed rule base R, the engine is the function

$$Q_{\tau,R} : \mathcal{X} \longrightarrow \mathcal{A}_\perp, \qquad Q_{\tau,R}(x) = Q_\tau(x, R). \tag{29}$$

It returns exactly one value: either one action or $\perp$. A tie never means "no match"; τ resolves the tie.

6.5 Associated Query Predicate

If a Boolean query predicate is useful, define

$$\text{Selected}_\tau : \mathcal{X} \times \mathfrak{R} \times \mathcal{A} \longrightarrow \mathbb{B}, \tag{30}$$

with

$$\text{Selected}_\tau(x, R, a) = \text{true} \quad \Longleftrightarrow \quad Q_\tau(x, R) = a, \qquad a \in \mathcal{A}. \tag{31}$$

Selected_τ is a predicate. Q_τ is the action-valued query function. They express the same selection semantics in relational and functional forms, respectively.

7 Core Properties

7.1 Determinism

Proposition 7.1 (Determinism)**.** *For every occurrence x, finite rule base R, and fixed selector τ, $Q_\tau(x, R)$ is unique.*

Proof. If $L_R(x)$ is empty, the result is $\perp$. Otherwise, finiteness gives a maximum specificity, so $B_R(x)$ is nonempty. The function τ selects one identifier from $\text{ids}(B_R(x))$. Identifier uniqueness determines exactly one winning rule, which contains exactly one action. □

7.2 Statelessness

Let $\mathcal{X}^*$ be the set of finite histories of event occurrences. Extend the fixed engine to a history-shaped interface by

$$G_{\tau,R}(h, x) = Q_{\tau,R}(x), \qquad h \in \mathcal{X}^*. \tag{32}$$

Then, for all histories $h_1, h_2 \in \mathcal{X}^*$ and current occurrences $x \in \mathcal{X}$,

$$\boxed{G_{\tau,R}(h_1, x) = G_{\tau,R}(h_2, x).} \tag{33}$$

The history argument is mathematically irrelevant. This history invariance is the precise meaning of statelessness in this note.

7.3 Payload-Extension Monotonicity

For every $c \in C$ and $p, p' \in \mathcal{P}_e$, adding extra payload bindings cannot invalidate an existing condition match:

$$p \subseteq p' \wedge \mu(c, p) = \text{true} \quad \Longrightarrow \quad \mu(c, p') = \text{true}. \tag{34}$$

This property belongs to condition matching. The selected action may still change after payload extension because the additional bindings can make new rules applicable. A new rule may have greater specificity than the previous winner or may enter an equal-specificity tie that τ resolves differently.

8 Courier Selection Example

Consider one event type,

$$\mathcal{E} = \{\texttt{ORDER\_RECEIVED}\}, \tag{35}$$

with the typed payload keys shown below.

Key	Value domain
`order_reference`	Σ^* for a character alphabet Σ
`delivery_type`	`{STANDARD, SAME_DAY}`
`vip`	`{false, true}`
`zone`	`{LOCAL, INTERNATIONAL}`

The delivery domain is reduced to the values needed by this compact example.

Let

$$\mathcal{A} = \{\texttt{SELECT\_LOCAL}, \texttt{SELECT\_SAME\_DAY}, \texttt{SELECT\_INTERNATIONAL}\}. \tag{36}$$

Use the following finite rule base. Every key omitted from a displayed condition set is absent from the corresponding partial map and therefore imposes no constraint.

Rule	Event	Condition set	σ	Action
r_1	ORDER_RECEIVED	zone=LOCAL, delivery_type=STANDARD, vip=true	6	SELECT_SAME_DAY
r_2	ORDER_RECEIVED	zone=LOCAL, delivery_type=STANDARD	4	SELECT_LOCAL
r_3	ORDER_RECEIVED	zone=LOCAL	2	SELECT_LOCAL
r_4	ORDER_RECEIVED	zone=LOCAL, delivery_type=SAME_DAY	4	SELECT_SAME_DAY
r_5	ORDER_RECEIVED	zone=INTERNATIONAL	2	SELECT_INTERNATIONAL

The local condition maps contain two specificity chains:

$$\text{condition}(r_3) \subset \text{condition}(r_2) \subset \text{condition}(r_1), \qquad \text{condition}(r_3) \subset \text{condition}(r_4). \tag{37}$$

Let

$$R = \{r_1, r_2, r_3, r_4, r_5\}, \qquad \{1, 2, 3, 4, 5\} \subseteq \mathcal{I}, \qquad \text{id}(r_j) = j \quad (1 \le j \le 5). \tag{38}$$

For the payload

$$\begin{aligned} p = \{&\texttt{order\_reference} \mapsto \texttt{0-1042}, \quad \texttt{delivery\_type} \mapsto \texttt{STANDARD}, \\ &\texttt{vip} \mapsto \text{true}, \quad \texttt{zone} \mapsto \texttt{LOCAL}\}, \end{aligned} \tag{39}$$

define the occurrence

$$x = (\texttt{ORDER\_RECEIVED}, p). \tag{40}$$

r_1, r_2, and r_3 match, with specificity scores 6, 4, and 2, respectively. Rules r_4 and r_5 do not match. Therefore

$$L_R(x) = \{r_1, r_2, r_3\}, \qquad B_R(x) = \{r_1\} \tag{41}$$

and

$$Q_{\tau,R}(x) = \texttt{SELECT\_SAME\_DAY}, \tag{42}$$

independently of the tie selector. The unused `order_reference` demonstrates that payload keys not mentioned by a condition set do not affect matching.

For a second payload,

$$p_{\mathrm{sd}} = \{\texttt{delivery\_type} \mapsto \texttt{SAME\_DAY}, \quad \texttt{vip} \mapsto \mathsf{false}, \quad \texttt{zone} \mapsto \texttt{LOCAL}\}, \tag{43}$$

let

$$x_{\mathrm{sd}} = (\texttt{ORDER\_RECEIVED}, p_{\mathrm{sd}}). \tag{44}$$

Here the absent `delivery_type` and `vip` keys in r_3 impose no constraints, so both r_3 and r_4 match. Rule r_4 is more specific:

$$L_R(x_{\mathrm{sd}}) = \{r_3, r_4\}, \qquad B_R(x_{\mathrm{sd}}) = \{r_4\}, \tag{45}$$

and therefore

$$Q_{\tau,R}(x_{\mathrm{sd}}) = \texttt{SELECT\_SAME\_DAY}. \tag{46}$$

The two occurrences demonstrate specificity chains of lengths three and two, respectively. These conditions contain no explicit-null wildcards, so each score is twice its number of concrete conditions.

9 Deliberate Limitations and Extensions

The definition is intentionally minimal.

- Conditions use conjunction, typed equality, and an explicit-null wildcard only. Inequalities, ranges, regular expressions, disjunctions, negation, and arbitrary domain predicates require explicit extensions.
- The fixed $2/1/0$ specificity weights are a syntactic priority policy. Other weights or logical refinement orders define alternative engines.
- The selector τ resolves only equal-specificity ties. It does not define scheduling, transaction priority, or action execution order.
- The rule base is finite and fixed during one query.
- External serialization and authoring syntax lie outside the mathematical core. For example, an adapter must preserve the distinction between JSON `null` and an omitted property.
- The returned action is an opaque value. Its execution may be stateful even though selection is not; such execution lies outside this model.

These restrictions make the core semantics suitable as a small reference oracle for an occurrence-local evaluator. Richer engines can add arguments and operators without changing what this core function means.

10 Minimal Definition

The complete engine reduces to the following equations:

$$
\begin{aligned}
&\mu(c,p) = \mathsf{true} \iff \forall k \in \mathrm{dom}(c):\ c(k) = * \vee (k \in \mathrm{dom}(p) \wedge c(k) = p(k) \in V_k)\,,\\
&\qquad \sigma(c) = 2n_{\mathrm{con}}(c) + n_{\mathrm{wild}}(c),\\
&\qquad L_R(x) = \{r \in R \mid \mathrm{Applicable}(r,x)\},\\
&\qquad B_R(x) = \operatorname*{arg\,max}_{r \in L_R(x)} \sigma(\mathrm{condition}(r))\,, \qquad L_R(x) \neq \varnothing,\\
&\quad w_{\tau,R}(x) \in B_R(x), \qquad \mathrm{id}\big(w_{\tau,R}(x)\big) = \tau(\mathrm{ids}(B_R(x)))\,,\\
&\quad Q_\tau(x,R) = \begin{cases} \bot, & L_R(x) = \varnothing,\\ \mathrm{action}\big(w_{\tau,R}(x)\big)\,, & L_R(x) \neq \varnothing. \end{cases}
\end{aligned}
\tag{47}
$$

The definitions of $B_R(x)$ and $w_{\tau,R}(x)$ apply only when $L_R(x) \neq \varnothing$.

Here μ is a predicate, σ is a numeric function, τ is a selector function, and Q_τ is an option-valued function. For fixed R and τ, $Q_{\tau,R}$ depends only on the current occurrence and is therefore stateless.

11 Conclusion

A minimal stateless ECA rule engine is not itself a predicate. It is a pure function from a current event occurrence and finite rule base to one action or $\bot$. Its Boolean condition predicate answers whether one condition set matches. Its specificity function weights concrete conditions, explicit-null wildcards, and omitted keys by two, one, and zero. Its fixed selector resolves only the maximally specific tie. Keeping these objects separate gives the model a precise type, makes determinism explicit, and locates specificity in the rule query rather than in condition truth.

The model preserves three distinct states. A condition-side `null` is an explicit wildcard; a payload-side `null` is a present null value; and an omitted key is absent. This distinction requires no normalization that discards authoring intent. Subject to that boundary, the definition is domain-independent, deterministic, and directly implementable as a reference semantics.

References

[1] D. R. McCarthy and U. Dayal, "The architecture of an active database management system," *Proceedings of the 1989 ACM SIGMOD International Conference on Management of Data*, pp. 215–224, 1989. doi:10.1145/66926.66946.

[2] S. Chakravarthy and D. Mishra, "Snoop: An expressive event specification language for active databases," *Data & Knowledge Engineering*, vol. 14, no. 1, pp. 1–26, 1994. doi:10.1016/0169-023X(94)90006-X90006-X).

[3] S. Chakravarthy, V. Krishnaprasad, E. Anwar, and S.-K. Kim, "Composite events for active databases: Semantics, contexts and detection," *Proceedings of the 20th International Conference on Very Large Data Bases*, pp. 606–617, 1994. VLDB paper.

[4] N. W. Paton and O. Díaz, "Active database systems," *ACM Computing Surveys*, vol. 31, no. 1, pp. 63–103, 1999. doi:10.1145/311531.311623.

[5] A. Paschke, "Reaction RuleML 1.0 for rules, events and actions in semantic complex event processing," *Rules on the Web: From Theory to Applications*, Lecture Notes in Computer Science, vol. 8620, pp. 1–21, 2014. doi:10.1007/978-3-319-09870-8_1.

[6] B. Berstel-Da Silva, "Formalizing both refraction-based and sequential executions of production rule programs," *Rules on the Web: Research and Applications*, Lecture Notes in Computer Science, vol. 7438, pp. 47–61, 2012. doi:10.1007/978-3-642-32689-9_5.

[7] J. Yen, "A principled approach to reasoning about the specificity of rules," *Proceedings of the Eighth National Conference on Artificial Intelligence*, pp. 701–707, 1990. AAAI paper.

[8] C. de Sainte Marie, G. Hallmark, and A. Paschke, eds., "RIF Production Rule Dialect (Second Edition)," W3C Recommendation, 5 February 2013. W3C Recommendation.